LABORATORY 5

Serialization & Networking

json_serializable, Equatable, HTTP and retries

Mobile and Embedded Computing · Dinu-Ștefan Rusu

LABORATORY 5

What you'll build today

Generate the boring half	Let json_serializable write fromJson and toJson so you never hand-parse a map again
Compare by value	Use Equatable so two instances holding the same data are actually equal
Fetch and map	Turn a live REST response into a typed List<Post> your UI can render
Fail well	Retry with exponential backoff and jitter instead of hammering a server that is down

This lab puts Lectures 5 and 6 to work. HTTP and code generation from Lecture 5; REST and the shape of an API from Lecture 6.

STARTING POINT

Where this lab's code lives

This lab does not extend the todo app. Create a new Flutter project inside your course repository.

flutter create lab5_networking, then work on a branch named lab5 in the same repository you have used all semester.

Two files carry the whole lab: lib/models/post.dart for Task I, and lib/api/post_client.dart for Task II. Keep them separate: the model must not know that HTTP exists.

Everything you call today is public and needs no API key, no account and no token. If a step asks you for credentials, you are on the wrong URL.

Files you will create

`lib/models/post.dart`

`lib/models/post.g.dart`

`generated, never edit it`

`lib/api/post_client.dart`

`lib/main.dart`

The whole lab is two files in your course repository. The model is pure Dart; only the client touches the network.

SETUP

The packages this lab needs

```
# Let pub write the current versions for you. Run this in the project folder:
flutter pub add json_annotation equatable dio
flutter pub add dev:build_runner dev:json_serializable
```

```
# pubspec.yaml: what they leave behind. The flutter: sdk entry stays as it is.
dependencies:
  json_annotation: ^4.9.0     # the annotations you write
  equatable: ^2.0.5           # value equality
  dio: ^5.7.0                 # or:  http: ^1.2.0
dev_dependencies:
  build_runner: ^2.4.13       # runs the generators, never ships in the app
  json_serializable: ^6.8.0   # the generator that writes post.g.dart
```

SETUP

The generated half, and how to make it

```
// lib/models/post.dart: top of the file
import 'package:json_annotation/json_annotation.dart';
import 'package:equatable/equatable.dart';

// The generated half. Same base name as this file, .g.dart suffix.
// It does not exist yet, so your editor underlines this line in red
// until you run the generator. That red is expected.
part 'post.g.dart';

# Generate it. Run again after every change to a model:
dart run build_runner build --delete-conflicting-outputs

# Or leave this running in a second terminal while you work:
dart run build_runner watch --delete-conflicting-outputs
```

--delete-conflicting-outputs is not optional. Without it the second run fails on the files the first one wrote.

TASK I

A model the generator can see

```dart
// lib/models/post.dart: the half you write by hand
@JsonSerializable()
class Post {
  const Post({required this.id, required this.authorId, required this.title});

  final int id;
  final String title;
  @JsonKey(name: 'userId')   // when the JSON key and the Dart field differ
  final int authorId;

  factory Post.fromJson(Map<String, dynamic> json) => _$PostFromJson(json);
  Map<String, dynamic> toJson() => _$PostToJson(this);
}
```

You write the annotations; the generator writes the parsing. _$PostFromJson and _$PostToJson live in post.g.dart and appear only after build_runner runs.

TASK I

Value equality with Equatable

```
// By default == in Dart is identity: same data, different object, not equal.
class Post extends Equatable {
  // ...the same fields and constructor as the previous slide...
  @override
  List<Object?> get props => [id, authorId, title];
}

// In main(), prove it:
final a = Post(id: 1, authorId: 1, title: 'hello');
final b = Post(id: 1, authorId: 1, title: 'hello');
print(a == b);          // false without Equatable, true with it
print({a, b}.length);   // 2 without, 1 with: a Set uses == and hashCode
```

A field you forget to list in props is a field == will ignore. That is the bug this package quietly introduces, so keep props complete.

TASK I

A model with generated JSON and equality

In lib/models/post.dart. Pick one model, Post or User, and stay with it for the rest of the lab.

1. Define the model class with at least four fields, all of them final
2. Annotate the class with @JsonSerializable() and add the matching part directive
3. Add the fromJson factory and the toJson method that delegate to the generated functions
4. Run build_runner and confirm post.g.dart appears containing _$PostFromJson and _$PostToJson
5. Rename one Dart field so it no longer matches its JSON key, and map it back with @JsonKey(name:)
6. Make the class extend Equatable and list every field in props
7. In main(), build two instances holding identical data and print the result of a == b

Hints

@JsonSerializable() on the class

part 'post.g.dart';

@JsonKey(name: 'userId')

extends Equatable, override props

dart run build_runner build

Done when

post.g.dart is generated and committed

a == b prints true

flutter analyze reports no issues

TASK II

The API you will call

```
# JSONPlaceholder: public, free, no key, no account.
GET  https://jsonplaceholder.typicode.com/posts      # 100 posts
GET  https://jsonplaceholder.typicode.com/posts/1    # a single post
GET  https://jsonplaceholder.typicode.com/nope       # 404, your failure case

# One element of the /posts response:
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident",
  "body": "quia et suscipit suscipit recusandae..."
}
```

Read the JSON before you write the model. The keys in that payload are exactly what @JsonKey has to match.

TASK II

Fetching and decoding

```dart
// lib/api/post_client.dart
final _dio = Dio(BaseOptions(
  baseUrl: 'https://jsonplaceholder.typicode.com',
  connectTimeout: const Duration(seconds: 5),
));

Future<List<Post>> fetchPosts() async {
  final res = await _dio.get('/posts');    // Dio throws on 4xx and 5xx
  final raw = res.data as List<dynamic>;
  return raw.map((j) => Post.fromJson(j as Map<String, dynamic>)).toList();
}
// Using package:http instead? jsonDecode(res.body) first, then the same map().
```

Decode once, at the edge. Past this function nothing in your app should ever see a Map<String, dynamic>.

TASK II

Retry with backoff and jitter

```
// import 'dart:math';
Future<T> withRetry<T>(Future<T> Function() op, {int maxAttempts = 5}) async {
  final rng = Random();
  for (var attempt = 0; ; attempt++) {
    try {
      return await op();
    } catch (e) {
      if (attempt >= maxAttempts - 1) rethrow;
      final backoff = 400 * (1 << attempt);   // 400, 800, 1600, 3200 ms
      final delay = backoff ~/ 2 + rng.nextInt(backoff ~/ 2 + 1);  // jitter
      debugPrint('attempt $attempt failed: $e, waiting ${delay}ms');
      await Future.delayed(Duration(milliseconds: delay));
    }
  }
}
```

Backoff spreads the retries out; jitter stops every client retrying in step.

TASK II

Fetch, map and survive a failing endpoint

In lib/api/post_client.dart, plus a screen in lib/main.dart that shows the result.

1. Fetch https://jsonplaceholder.typicode.com/posts with dio or package:http
2. Map the response onto a List<Post> using the fromJson generated in Task I
3. Render those posts in a ListView so a successful fetch is visible in the running app
4. Wrap the call in a retry helper that gives up after a fixed number of attempts
5. Make the delay grow exponentially between attempts, starting from a few hundred milliseconds
6. Add random jitter to each delay so two clients retrying never line up
7. Point the client at an invalid endpoint and log every attempt together with its delay

Hints

Dio.get / http.get

Future.delayed(Duration(...))

Random().nextInt(...) from dart:math

1 << attempt doubles the delay

debugPrint, not print

Done when

Posts from the live API render on screen

The log shows five attempts with growing, non-identical delays

That log screenshotted into /screenshots

TROUBLESHOOTING

When it goes wrong

	What it means	What to do
Target of URI hasn't been generated	You have not run the generator yet	dart run build_runner build --delete-conflicting-outputs
Conflicting outputs were detected	Generated files from an earlier run are in the way	Add --delete-conflicting-outputs to the command
Undefined name '_$PostFromJson'	The part directive is missing or names the wrong file	part 'post.g.dart'; must match this file's name exactly
a == b still prints false	props is incomplete, or the class does not extend Equatable	List every field in props and rerun
List<dynamic> is not a subtype	You passed the decoded body straight into fromJson	Cast each element to Map<String, dynamic> while mapping

Rerun the generator after every model change. Most of these are one stale post.g.dart in five disguises.

SUBMISSION

How to hand this in

Everything goes to your GitHub repository. No Word documents, no email, no Teams upload.

Commit your work to a branch named lab5, open a pull request, and merge it once it works.

Screenshots asked for in a task go in a /screenshots folder in the same repository.

Your commit history is part of the assessment. Commit as you go, not once at the end.

Checklist

- code committed and pushed
- screenshots where asked
- app builds and runs
- pull request merged

Expected in /screenshots: the console output showing a == b is true, and the retry log with its growing delays against the invalid endpoint.

Commit post.g.dart along with everything else, because the grader needs to see what the generator produced.

Wrapping up

Recap

You write annotations; build_runner writes fromJson and toJson

Equatable gives value equality and a matching hashCode from props

Backoff spreads retries out in time; jitter keeps clients from syncing up

Before next lab

Push branch lab5 and open the pull request

Screenshots of the retry log in /screenshots

Bring questions: the project needs every piece of this

Read more

pub.dev/packages/json_serializable

pub.dev/packages/equatable

jsonplaceholder.typicode.com